560. Subarray sum equals k

```java
class Solution {
    public int subarraySum(int[] nums, int k) {
        int cnt=0,sum=0;
        Map<Integer,Integer>map=new HashMap<>();
        map.put(0,1);
        for(int i=0;i<nums.length;i++){
            sum+=nums[i];
            if(map.containsKey(sum-k)){
                cnt+=map.get(sum-k);
            }
            map.put(sum,map.getOrDefault(sum,0)+1);
        }
        return cnt;
    }
}
```

56. Merge Intervals

```java
ava  Auto
class Solution {
    public int[][] merge(int[][] intervals) {
        if(intervals.length<=1)
            return intervals;
        Arrays.sort(intervals,Comparator.comparingInt(i -> i[0]));
        List<int[]> res=new ArrayList<>();
        int[] newinterval=intervals[0];
        res.add(newinterval);
        for(int[] interval:intervals){
            if(interval[0]<=newinterval[1]){
                newinterval[1]=Math.max(newinterval[1],interval[1]);
            }else{
                newinterval=interval;
                res.add(newinterval);
            }
        }
        return res.toArray(new int[res.size()][]);
    }
}
```

703. Kth largest element in a Stream

Code

ava Auto

```java
class KthLargest {

    PriorityQueue<Integer>minheap;
    int k;
    public KthLargest(int k, int[] nums) {
        minheap=new PriorityQueue<>();
        this.k=k;
        for(int num:nums){
            add(num);
        }
    }

    public int add(int val) {
        if(minheap.size()<k||minheap.peek()<val){
            minheap.add(val);
            if(minheap.size()>k){
                minheap.remove();
            }
        }
        return minheap.peek();
    }
}

/**
```

Remove Cycle in Linked List

```java
/*
class Node
{
    int data;
    Node next;
}
*/

class Solution {
    public static void removeLoop(Node head) {
        Node p1=head;
        Node p2=head;
        while(p2!=null &&p2.next!=null){
            p2=p2.next.next;
            p1=p1.next;
            if(p1==p2){
                p1=head;
                if(p1==head &&p2==head){
                    while(p1.next!=head) p1=p1.next;
                    p1.next=null;
                    return;
                }
                while(p1.next!=p2.next){
                    p1=p1.next;
                    p2=p2.next;
                }
                p2.next=null;
                return;
            }
        }
    }
}
```

236. LCA of a Binary Tree

Code

Java Auto

```java
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if(root==null||root==p||root==q) return root;
        TreeNode left=lowestCommonAncestor(root.left,p,q);
        TreeNode right=lowestCommonAncestor(root.right,p,q);
        if(left!=null&&right!=null) return root;
        if(left==null) return right;
        else return left;
    }
```

1091. Shortest path in a binary matrix

```java
class Solution {
    public int shortestPathBinaryMatrix(int[][] grid) {
        if(grid[0][0]==1) return -1;
        int m=grid.length;
        int n=grid[0].length;
        Queue<int[]> q=new LinkedList<>();
        q.add(new int[]{0,0,1});
        grid[0][0]=1;
        int[][] dir={{0,1},{1,0},{0,-1},{-1,0},{1,1},{1,-1},{-1,1},{-1,-1}};
        while(!q.isEmpty()){
            int size=q.size();
            while(size-- >0){
                int[] point=q.poll();
                if(point[0]==m-1 && point[1]==n-1){
                    return point[2];
                }
                for(int[] d:dir){
                    int r=point[0]+d[0];
                    int c=point[1]+d[1];
                    if(r>=0 &&c>=0 && r<m && c<n && grid[r][c]==0){
                        q.add(new int[]{r,c,point[2]+1});
                        grid[r][c]=1;
                    }
                }
            }
        }
        return -1;
    }
}
```

322. Coin Change

</> Code

ava ▾ 🔒 Auto

```java
class Solution {
    public int coinChange(int[] coins, int amount) {
        if(amount==0) return 0;
        int[] dp=new int[amount+1];
        Arrays.fill(dp,Integer.MAX_VALUE);
        for(int i:coins){
            if(i>amount) continue;
            dp[i]=1;
            for(int j=1;j<=amount;j++){
                if(dp[j]!=Integer.MAX_VALUE&&j+i<=amount)
                    dp[j+i]=Math.min(dp[i+j],dp[i]+dp[j]);
            }
        }
        return dp[amount]!=Integer.MAX_VALUE ?dp[amount]:-1;
    }
}
```